

CIENCIA DE DATOS (TA047)

Cátedra Dr. Ing. Rodríguez - 1°C 2025

Trabajo práctico 2 – Grupo 3: Los sistemáticos

REPORTE

Integrantes:

- **Arias Rafaela Pilar**, Padrón: 112272,
Email: rparias@fi.uba.ar.
- **Barcala Roca Ramiro**, Padrón: 112478,
Email: rbarcala@fi.uba.ar.
- **Ienco, Lara Eliana**, Padrón: 111921,
Email: lienco@fi.uba.ar.
- **Lema Mora**, Padrón: 111831,
Email: molema@fi.uba.ar.

Fecha de entrega: 26/06/25

ÍNDICE

Dataset: Críticas cinematográficas.....	2
Introducción.....	2
Modelos.....	3
<i>Random Forest.....</i>	<i>3</i>
<i>XGBoost.....</i>	<i>4</i>
<i>Bayes Naïve.....</i>	<i>5</i>
<i>Redes neuronales.....</i>	<i>6</i>
<i>Stacking (Ensamble Personalizado).....</i>	<i>8</i>
Cuadro de resultados.....	10
Conclusiones.....	10
Cuadro de tiempo dedicado.....	12

Dataset: Críticas cinematográficas

Introducción

El objetivo de este ejercicio es analizar porciones de texto –críticas cinematográficas– e identificar si el sentimiento es positivo o negativo. Para ello es necesario construir distintos modelos de clasificación, capaces de analizar texto en lenguaje natural y asignarle un sentimiento binario, positivo o negativo.

La descarga del dataset fue mediante Kaggle, donde los datos se hallaban divididos en train y test. El conjunto de train tiene 50.000 filas y el de test tiene 8599 filas. En cuanto a las columnas, el conjunto train tiene tres columnas: ID, review_es y sentimiento. Por su parte, el conjunto test tiene dos columnas: ID y review_es.

Dentro del análisis previo realizado al dataset, verificamos que no tuviera ninguna fila con algún parámetro NaN o filas duplicadas y observamos la distribución de la clase positivo y negativo, para evitar desbalances de clases. La distribución fue 50 / 50 y tampoco había valores NaN o filas duplicadas, por lo que no fue necesario aplicar ninguna técnica como Under u Oversampling o detección de valores faltantes.

Antes de procesar los textos, verificamos si todos los registros del dataset estaban efectivamente en español. Para ello, aplicamos un detector de idioma sobre la columna de reseñas utilizando la librería langid. Esta función nos permitió identificar el idioma predominante de cada texto tanto en el conjunto de entrenamiento como en el de prueba. Al realizar este análisis, observamos que el dataset de entrenamiento contenía reseñas en cinco idiomas diferentes: principalmente en español (es), pero también en inglés (en), gallego (gl), portugués (pt) e indonesio (id). En el conjunto de prueba, además del español, aparecieron textos en gaélico (ga) y latín (la). Dado que el objetivo del proyecto es trabajar exclusivamente con textos en español, filtramos el dataset de entrenamiento para conservar únicamente aquellas reseñas cuyo idioma fue identificado como es. Esto redujo el conjunto de entrenamiento a 48.179 ejemplos (originalmente eran 50.000, por lo que no perdimos tanta información).

Procedimos con la Tokenización, creando una serie de funciones para poder limpiar el texto correctamente. Nos aseguramos de: pasar el texto a minúsculas, añadir espacio después de puntos y comas si es que no hay para evitar que se junten palabras y eliminar caracteres extraños. Además, creamos una **stoplist** y establecimos como lenguaje al español, para poder filtrar preposiciones, pronombres y artículos que no aportan demasiado al sentimiento de la crítica. Otra

técnica de filtrado que aplicamos al crear la **bag of words** fue la de descartar palabras que fueran poco o muy frecuentes.

Además incorporamos la **lematización**, utilizando la librería spaCy y su modelo en español. Este proceso consiste en reducir cada palabra a su forma canónica o raíz (por ejemplo, "bailando" se transforma en "bailar"), lo cual permite unificar variantes gramaticales que, en el análisis de sentimiento, no suelen aportar diferencias significativas. La lematización se aplicó luego del proceso de limpieza, antes de vectorizar los textos con TF-IDF. En Redes no lo usamos con TF IDF ya que no hizo falta, pues BERT tokeniza, le pone IDS a los tokens, y luego usa embeddings como método de vectorización.

Luego del preprocesamiento textual y la vectorización de las críticas, implementamos la función *evaluar*, que es similar a una serie de funciones que aplicamos que utilizamos en el Trabajo Práctico 1. Esta función nos permite observar los resultados de las métricas en train y test, para poder saber con más certeza cómo se desempeña el modelo. También implementamos matrices de confusión para cada modelo así observábamos directamente dónde podría mejorarse.

Modelos

Random Forest

Utilizamos GridSearchCV, aplicando K-Fold Cross Validation con 5 folds. Priorizamos trabajar con F1 en el scoring, ya que fue estipulado como requisito en la competencia de Kaggle. Los hiperparámetros sobre los que GridSearchCV buscó los mejores parámetros son:

- **criterion**: criterio de medida para las divisiones ['gini'].
- **n_estimators**: cantidad de árboles en el bosque [200, 300, 400].
- **max_depth**: cuán profundo puede crecer cada árbol individual dentro del bosque [5, 10].
- **min_samples_split**: cantidad mínima de muestras para dividir un nodo [5, 10].
- **min_samples_leaf**: cantidad mínima de muestras en una hoja del árbol [2, 4, 6].

Probamos con distintos vectorizadores (Count Vectorizer y TFID-Vectorizer) y el que mejor resultado obtuvo fue el segundo, aunque sus diferencias fueron mínimas.

Con respecto a los hiperparámetros, probamos con muchas combinaciones distintas, pero no logramos aumentar el F1 en Kaggle. Cuando creíamos que iba a

subir porque reducíamos la tendencia al overfitting del modelo, era cuando más bajaba en Kaggle. Honestamente teníamos expectativas más elevadas para el Random Forest.

XGBoost

Para este modelo aplicamos el LabelEncoder para el conjunto de entrenamiento, pues XGBoost no acepta etiquetas de clase tipo String, entonces transformamos las clases a valores numéricos. Además, planteamos un espacio amplio de búsqueda, por lo que, en un principio, aplicamos RandomizedSearchCV para hallar hiperparámetros.

Pero luego recordamos una recomendación que nos comentaron al exponer el TP 1: en vez de aplicar únicamente el RandomizedSearchCV, podíamos utilizarlo y almacenar una cierta cantidad de combinaciones de parámetros –por ejemplo, el top 10 mejores combinaciones– y sobre eso aplicar el GridSearchCV. Por lo que elegimos obtener el top 5 y aplicar GridSearchCV sobre ellos, para quedarnos con la mejor combinación de todas. Utilizamos K-Fold Cross Validation con 4 folds para ambas búsquedas.

Los hiperparámetros sobre los que RandomizedSearchCV buscó el top 5 son:

- `n_estimators`: número total de árboles [150, 250, 300].
- `max_depth`: cuánta es la profundidad máxima de cada árbol (3, 4, 5).
- `learning_rate`: cuánto aprende de los errores anteriores (0.05, 0.01).
- `subsample`: qué porcentaje de los datos ve cada árbol [0.5, 0.7, 0.8]
- `colsample_bytree`: qué porcentaje de las columnas (features) ve cada árbol (0.6, 0.8).
- `gamma`: divide el árbol según si la mejora es buena (0, 5, 6).
- `'reg_alpha'`: aplica una penalización absoluta a los pesos del modelo [0, 0.05, 0.1],
- `'reg_lambda'`: aplica una penalización cuadrática a los pesos [0.5, 1, 1.5]

El top 5 es:

1. `{'subsample': 0.5, 'reg_lambda': 1.5, 'reg_alpha': 0.05, 'n_estimators': 300, 'max_depth': 5, 'learning_rate': 0.1, 'gamma': np.float64(4.0), 'colsample_bytree': 0.8}`
2. `{'subsample': 0.7, 'reg_lambda': 1.5, 'reg_alpha': 0, 'n_estimators': 300, 'max_depth': 5, 'learning_rate': 0.1, 'gamma': np.float64(0.0), 'colsample_bytree': 0.6}`
3. `{'subsample': 0.5, 'reg_lambda': 1.5, 'reg_alpha': 0.05, 'n_estimators': 250, 'max_depth': 5, 'learning_rate': 0.1, 'gamma': np.float64(4.0), 'colsample_bytree': 0.6}`

4. {'subsample': 0.7, 'reg_lambda': 0.5, 'reg_alpha': 0.05, 'n_estimators': 300, 'max_depth': 4, 'learning_rate': 0.1, 'gamma': np.float64(5.0), 'colsample_bytree': 0.8}
5. {'subsample': 0.5, 'reg_lambda': 1.5, 'reg_alpha': 0.05, 'n_estimators': 250, 'max_depth': 4, 'learning_rate': 0.1, 'gamma': np.float64(4.0), 'colsample_bytree': 0.8}

En este caso también esperábamos un mejor rendimiento en Kaggle, teníamos más expectativas que con el RandomForest. Probamos muchas búsquedas distintas: con varios hiperparámetros distintos, solo haciendo GridSearch o RandomSearch e incluso probamos incluyendo Early Stopping. Probamos con y sin lematización, con los distintos vectorizadores... pero no logramos obtener un resultado satisfactorio.

Bayes Naïve

Para entrenar el modelo Naive Bayes comparamos tres enfoques de clasificación, combinando distintas estrategias de representación del texto (TF-IDF y con CountVectorizer, considerando en ambos unigramas, bigramas y trigramas) y dos modelos (Naive Bayes y un clasificador Dummy). Al igual que en el XGBoost, utilizamos el LabelEncoder para el conjunto de entrenamiento, convertimos las etiquetas categóricas a valores numéricos dado que el modelo no acepta strings como etiquetas.

Comenzamos entrenando un clasificador Multinomial Naive Bayes utilizando los vectores generados por TF-IDF. Evaluamos al clasificador entrenado buscando los hiperparámetros con GridSearchCV con 10 folds, considerando al F1, para ver cómo generalizaba el modelo.

El segundo enfoque fue entrenar un clasificador Dummy, con la vectorización TF-IDF para utilizarlo como referencia al comparar el modelo. Como predice al azar según la distribución de clases, nos sirve como una referencia mínima para comparar el rendimiento del modelo real.

Proseguimos entrenando el modelo Naive Bayes utilizando como entrada la representación del texto generada con CountVectorizer. Para ello, buscamos los hiperparámetros con GridSearchCV con 10 folds, considerando la métrica F1 macro.

Finalmente, mostramos en formato de tabla el desempeño promedio de F1 en cada modelo:

<u>Modelo</u>	<u>F1-Score</u>
Bayes Naïve con TF-IDF	0.8816

Bayes Naïve con Dummy	0.5047
Bayes Naïve con CountVectorizer	0.8816

Notamos que el F1 macro promedio es igual usando CountVectorizer y TF-IDF. Utilizamos el coseno de similitud para ver el resultado y nos dio 0.8995, lo que indica que los vectores son casi iguales, y tiene sentido que están generando representaciones muy parecidas ya que el modelo es el mismo (MultinomialNB), el espacio de búsqueda es igual (alpha, fit_prior) y usamos los mismos parámetros para ambos vectorizadores (como ngram_range, min_df, max_df). Finalmente, optamos por implementar la variante con TF-IDF porque no solo cuenta las palabras, sino que les da más importancia a las que son realmente útiles para distinguir los textos.

Redes neuronales

Construimos nuestra red neuronal utilizando Pytorch por accidente. La implementación con Pytorch fue sencilla y lineal, y la mayoría de la exploración se hizo investigando qué tipo de métodos de regularización usar, qué valores asignarle a los distintos hiperparámetros y cómo mejorar el preprocesamiento. Cuando nos percatamos de que tendríamos que haberlo hecho aplicando Keras y Tensor Flow, quisimos aplicarlo a nuestra implementación pero resultó más complejo de lo que esperábamos. Lamentablemente, nos dimos cuenta cerca de la fecha de entrega, por lo que lo comentamos por Slack y agradecemos profundamente que nos hayan permitido continuar con nuestra implementación inicial.

Además, utilizamos transformers con el modelo BERT, de la plataforma HuggingFace. La versión de BERT que elegimos es una preentrenada para el idioma español, desarrollada por la Universidad Católica de Chile. El modelo es: **dccuchile/bert-base-spanish-wwm-cased** y fue entrenado con Whole Word Masking, que es una técnica que selecciona aleatoriamente un porcentaje de los tokens en una oración y los reemplaza con el token "MASK", haciendo que el modelo deba adivinar qué había ahí.

Respecto a los distintos métodos de regularización, inicialmente implementamos Dropout, que va apagando activaciones aleatoriamente. Comenzamos con un Dropout del 30% pero el modelo tenía mucho sobreentrenamiento, pues el Train y Validación daban métricas alrededor del .96, 0.97 e incluso 0.98 y en Kaggle el F1 apenas llegaba al 0.80. Por lo que luego probamos con Early Stopping, que no fue tan bueno como esperábamos. Finalmente, decidimos darle una segunda oportunidad al Dropout, usando uno más

alto (del 50%). Este Dropout ayudó a reducir el overfitting y a reducir las métricas en el entrenamiento y validación.

En cuanto a los distintos hiperparámetros la búsqueda fue manual y usamos siguientes:

- **MAX_LEN:** la longitud máxima de tokens que se le permite a cada texto luego de ser tokenizado con BERT. Si es más corto, se completa con padding y si es más largo, se trunca. Comenzamos con un valor elevado, de 256. Fuimos probando varios valores distintos: 192, 128, 100, 64... pero el que mejor resultó fue el de **128**, que resultó en una ejecución más rápida del modelo y además ayudó a evitar el sobreentrenamiento. No es ni muy elevado ni un número muy pequeño
- **BATCH_SIZE:** el tamaño del batch, que es la cantidad de reviews que se procesan juntas en una sola pasada por el modelo. El valor inicial fue **16**, que luego probamos bajándolo a 8 y subiéndolo a 18. Si el batch es chico, el modelo es más estable pero mucho más lento, y si es más grande es más rápido pero en nuestro caso solo fomentaba el sobreentrenamiento.
- **NCLASSES:** el número de clases posibles que va a predecir el modelo. En este caso no exploramos y siempre se mantuvo en **2**, porque era justamente lo que necesitábamos.
- **EPOCHS:** es el número de épocas, cuántas veces se pasa todo el dataset de entrenamiento por la red. En un principio elegimos **3** porque nos pareció un número adecuado, pero luego confirmamos que nuestra elección fue la correcta, ya que al probar con 4 no ayudaba a limitar el sobreentrenamiento, y 2 era demasiado poco.
- **L_RATE:** es la tasa de aprendizaje, cuánto va aprendiendo el modelo. En nuestro caso, en un inicio lo establecimos como $3e-5$, que es 0.00003. Luego experimentamos con 0.1, que fue demasiado. Entonces nos quedamos cerca del valor $3e-5$, probando con $1e-5$ y finalmente con $2e-5$. El que mejor se adecuó a nuestro modelo para evitar el sobreentrenamiento fue **$2e-5$** .

Construimos una clase personalizada en PyTorch (BERTSentimentClassifier), que incorpora a BERT como capa base, seguida de una capa de dropout y una capa lineal de salida para clasificación binaria. Además, entrenamos el modelo utilizando AdamW como optimizador y CrossEntropyLoss como función de pérdida.

Para poder organizar la carga de datos en los batches usamos un DataLoader con el cual dividimos el dataset en batches para cargar los datos en partes más pequeñas y luego hicimos shuffling para mezclar los datos en cada época así que el modelo no aprende por orden. Esta implementación nos permitió cargar los datos en paralelo, usando varios procesos para acelerar la carga.

Stacking (Ensamble Personalizado)

Elegimos implementar el Stacking, basado en Regresión Logística, RandomForest, MultinomialNB y XGBoost como metamodelo. El ensamble fue uno de los modelos que nos llevó más tiempo y donde más exploramos, ya que fuimos probando muchas combinaciones de modelos distintas.

Comenzamos con RandomForest, XGBoost, SVM y Regresión Logística como metamodelo, pero SVM nos resultaba un modelo algo lento y creíamos que el ensamble tenía aún más potencial. Luego modificamos el SVM por el MultinomialNB, pero aún no nos sentíamos satisfechos por lo que incluimos Embeddings en nuestro trabajo. Los Embeddings nos resultaron una alternativa interesante, pero el MultinomialNB no funcionaba correctamente por lo que lo reemplazamos por Adaboost.

Luego consideramos que quizás era más óptimo si se usaba un modelo más potente como metamodelo, por lo que intercambiamos la Regresión Logística por el XGBoost, quedándonos RandomForest, Regresión Logística, Adaboost y XGBoost como metamodelo.

Pero todavía no estábamos satisfechos con los resultados: en general, el mejor F1 nos daba 0.80 y las métricas de validación no eran lo que esperábamos. Además, tenían mucha diferencia con Kaggle, que el F1 daba 0.70.

```
Mejores hiperparámetros: {'ada__n_estimators': 132, 'final_estimator': 'logit', 'n_estimators': 100, 'random_state': 42, 'stacking__estimators': ['logit', 'ada', 'xgb'], 'stacking__meta_model': 'xgb', 'stacking__num_models': 3, 'stacking__weights': [0.5, 0.5, 0.5]}  
Mejor score F1: 0.8082116260100616
```

Reporte en validación:				
	precision	recall	f1-score	support
negativo	0.82	0.82	0.82	5000
positivo	0.82	0.82	0.82	5000
accuracy			0.82	10000
macro avg	0.82	0.82	0.82	10000
weighted avg	0.82	0.82	0.82	10000

Seguíamos creyendo en la potencia del XGBoost como metamodelo, pero queríamos probar otras alternativas además de Embeddings, por lo que quitamos el Adaboost y volvimos a agregar el MultinomialNB. En vez de los Embeddings, aplicamos el TF-IDF y pulimos algunos hiperparámetros.

Finalmente, quedamos satisfechos tras ver cómo subieron las métricas drásticamente: el mejor F1 pasó a ser 0.8963 y en Kaggle dio 0.75118. Además, redujimos la diferencia entre los conjuntos de entrenamiento y validación.

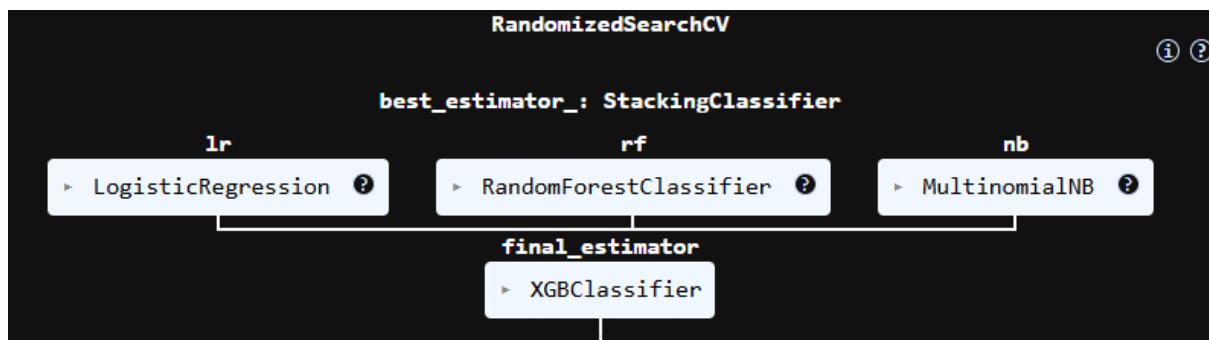
Los hiperparámetros finales sobre los que se hizo RandomizedSearchCV son:

'final_estimator__subsample': uniform(0.6, 0.3), 'final_estimator__colsample_bytree': uniform(0.6, 0.2)

- 'lr__C': cuánto se evitan pesos grandes (0.1, 0.5).
- 'rf__n_estimators': número total de árboles (100, 125, 150).
- 'rf__max_depth': cuán profundo puede crecer cada árbol individual dentro del bosque (5, 7, 9).
- 'final_estimator__learning_rate': cuánto aprende de los errores anteriores (0.07, 0.1).
- 'final_estimator__max_depth': cuánta es la profundidad máxima de cada árbol (3, 4).
- 'final_estimator__n_estimators': número total de árboles (80, 100, 120).
- 'final_estimator__subsample': qué porcentaje de los datos ve cada árbol (0.6, 0.3).
- 'final_estimator__colsample_bytree': qué porcentaje de las columnas ve cada árbol (0.6, 0.2).

Siendo **lr**: Regresión Logística, **rf**: RandomForest y **final_estimator**: XGBoost.

Luego de realizar RandomSearchCV con 3 folds y n_iter = 15, los mejores hiperparámetros fueron:



LogisticRegression(C=np.float64(0.5849549260809972), max_iter=300,
random_state=42)

RandomForestClassifier(max_depth=7, n_estimators=125, random_state=42)

MultinomialNB()

XGBClassifier(colsample_bytree=np.float64(0.6918497783931734),
learning_rate=np.float64(0.1033708611139022), max_depth=4, n_estimators=120,
random_state=42)

Resultando con un promedio de F1 en validación cruzada igual a 0.8970.

Luego, observamos las matrices de confusión del conjunto entrenamiento y validación para comparar el rendimiento con números reales y claros más allá de las métricas, y para ver dónde podría estar fallando.

Cuadro de resultados

Medidas de rendimiento del conjunto TEST:

<u>Modelo</u>	<u>F1-Test</u>	<u>Precision Test</u>	<u>Recall Test</u>	<u>Accuracy Test</u>	<u>Kaggle</u>
RandomForest	0.8374	0.8074	0.8696	0.8314	0.72300
XGBoost	0.8529	0.8451	0.8609	0.8518	0.71002
Bayes Naïve	0.8800	0.8672	0.8931	0.8784	0.74820
Redes Neuronales	0.9335	0.8917	0.9302	0.9302	0.81760
Stacking	0.8963	0.8903	0.9025	0.8958	0.75118

Parámetros elegidos del RandomForest: {'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 400}

Parámetros elegidos del XGBoost: {'colsample_bytree': 0.8, 'learning_rate': 0.12, 'max_depth': 6, 'n_estimators': 350, 'subsample': 0.5}

Parámetros elegidos del Bayes Naïve: {'alpha': 0.5, 'fit_prior': True}

Parámetros elegidos de la Red Neuronal: {MAX_LEN = 128, BATCH_SIZE = 16, NCLASSES = 2, EPOCHS = 3, L_RATE = 2e-5, DROPOUT= 0.5}

Parámetros elegidos del Stacking: {'final_estimator__colsample_bytree': np.float64(0.6918497783931734), 'final_estimator__learning_rate': np.float64(0.1033708611139022), 'final_estimator__max_depth': 4, 'final_estimator__n_estimators': 120, 'final_estimator__subsample': np.float64(0.6061753482887408), 'lr_C': np.float64(0.5849549260809972), 'rf__max_depth': 7, 'rf__n_estimators': 125}

Conclusiones

A modo de conclusión, destacamos que el trabajo fue interesante ya que aplicamos muchos más modelos distintos para predecir el sentimiento. Fue divertido e interesante explorar y tratar de refinarlos lo más posible. Somos uno de los grupos con la mayor cantidad de submissions –61– porque realmente quisimos probar de

todo. Incluso llegamos a sentir que 5 submissions por día era poco, y rápidamente identificamos que Kaggle considera un día nuevo a partir de las 9 p.m.

El análisis exploratorio resultó clave para lograr mejorar los modelos, ya que sin éste era como que les “faltaba algo” para proporcionar buenos resultados. Además de la lematización, lo que realmente cambió el rendimiento fue la incorporación de Langid para detectar y filtrar los textos que no estaban en español. No esperábamos encontrarnos con tantos textos en idiomas tan variados, y su eliminación cambió mucho el desempeño de los modelos. Lo implementamos en todos los modelos ya que si llegaba a faltar en alguno no sería el mismo criterio de evaluación, porque un modelo aprendería patrones en español y otro en español y gallego, portugués, indonesio... entonces, si lo agregábamos –que es lo que nos parecía correcto–, debía estar en todos los modelos.

Por otro lado, el modelo con mejor desempeño en F1 dentro de Test fue Redes neuronales, seguido por el Stacking. En Kaggle el que superó a todos los demás modelos fue Redes Neuronales, con un 0.81760 y le siguió Stacking con 0.75188.

El modelo más fácil y rápido de entrenar fue el Bayes Naïve. Fue un modelo que se ejecutó rápido, y tras evaluar los tres enfoques, elegir el correcto y agregar la filtración por español nos sorprendió con su puntaje que pasó de 0.72824 a 0.74820. Su desempeño no estuvo nada mal para lo que resultó ser el tiempo de ejecución, por lo cual nos pareció que sí fue útil.

Honestamente nos habría gustado tener aún más tiempo para poder seguir intentando mejorar el RandomForest y el XGBoost, ya que su rendimiento nos dejó insatisfechos. Teníamos expectativas más altas, no tantas como con Redes o el Stacking, pero un 0.71 y 0.72 nos resultó llamativo y decepcionante. Pero llegó un punto en el que probamos de todo y no sabíamos qué más intentar. Quizás podríamos haber aplicado algo más en el preprocesamiento, pero la realidad es que filtramos por idiomas, usamos los distintos vectorizadores, probamos aplicar Embeddings, ver si había desbalance... pero aún así disfrutamos del trabajo, sobre todo al implementar los otros tres modelos que sí fueron mostrando resultado mientras íbamos explorando y puliéndolos.

Cuadro de tiempo dedicado

Integrante	Tarea	Prom. Hs. Semana
Ramiro Barcala Roca	Embeddings, Redes Neuronales	6
Rafaela Pilar Arias	XGBoost, Ensamble (Stacking), Redes Neuronales, Informe	10
Mora Lema	Feature Engineering, Tokenización, RandomForest, XGBoost, Bayes Naïve, Ensamble (Stacking), Informe	14
Lara Eliana Ienco	Preprocesamiento del dataset, Tokenización, Bayes Naïve, XGBoost	8